

BÁO CÁO: TỐI ƯU HÓA QUY TRÌNH NGHIÊN CỨU KHOA HỌC DỮ LIỆU BẰNG PROMPT ENGINEERING

Sinh viên thực hiện: Nguyễn Hạnh Thùy Dương

Mã Sinh Viên: 25024088

Trường: Đại học Công nghệ - ĐHQGHN

Chủ đề: Nhập môn Lập trình hướng đối tượng (OOP)

I. Lựa chọn và phân tích tác vụ học tập

1. Tác vụ 1: Hệ thống hóa sự chuyển đổi tư duy (Paradigm Shift Summary)

- Mục tiêu:** Nhận diện tại sao cách tiếp cận cũ (dữ liệu tách rời hàm) lại thất bại khi quy mô dự án tăng lên.
- Thách thức:** AI dễ sa vào việc liệt kê định nghĩa thay vì giải thích "triết lý" đằng sau sự thay đổi này.

2. Tác vụ 2: Giải thích khái niệm "Tính đóng gói" (Encapsulation) và "Che giấu dữ liệu" (Data Hiding)

- Mục tiêu:** Hiểu rõ cơ chế bảo vệ trạng thái nội bộ của đối tượng thông qua private và public.
- Thách thức:** Cần sự kết nối giữa lý thuyết trừu tượng và cú pháp C++ thực tế để tránh việc hiểu sai bản chất.

3. Tác vụ 3: Thiết kế công cụ kiểm tra tư duy đối tượng (Class và Object)

- Mục tiêu:** Phân biệt rạch ròi giữa "khuôn mẫu" (Class) và "thực thể" (Object/Instance).
- Thách thức:** Tạo ra các câu hỏi tình huống thực tế thay vì chỉ hỏi lý thuyết suông.

II. Xây dựng các phiên bản Prompt (Tiêu chí 2)

1. Tác vụ 1: Hệ thống hóa sự chuyển đổi tư duy

- Prompt Cơ bản:** "Hãy tóm tắt bài giảng Introduction to OOP này."
- Prompt Cải tiến:** "Tóm tắt bài giảng về OOP. Hãy tập trung vào sự khác biệt giữa Lập trình thủ tục và OOP. Trình bày dưới dạng danh sách so sánh."

- **Prompt Nâng cao (Role + Chain-of-Thought):**

Role: Bạn là một Senior Software Architect chuyên về hệ thống hướng đối tượng. **Task:** Hãy phân tích sự chuyển đổi từ Procedural sang OOP dựa trên nội dung bài giảng. **Chain-of-Thought:**

1. Hãy bắt đầu bằng việc nêu nhược điểm của việc dữ liệu và hàm tách rời trong lập trình thủ tục.
2. Giải thích cách OOP "đóng gói" chúng lại thành một thực thể duy nhất.
3. Lấy ví dụ về lớp Student (với các thuộc tính như name, ID và phương thức study()) để minh họa. **Constraint:** Sử dụng ngôn ngữ chuyên sâu nhưng súc tích, định dạng Markdown rõ ràng.

2. Tác vụ 2: Giải thích tính đóng gói (Encapsulation)

- **Prompt Cơ bản:** "Giải thích tính đóng gói trong OOP."
- **Prompt Cải tiến:** "Giải thích Encapsulation là gì và cho ví dụ bằng code C++. Tại sao chúng ta cần Getter và Setter?"
- **Prompt Nâng cao (Few-shot + Analogy):**

Role: Bạn là một giảng viên PPLLT nổi tiếng với cách dạy trực quan.

Context: Sinh viên đang gặp khó khăn trong việc hiểu tính đóng gói là gì và tại sao phải để thuộc tính là private.

Technique: Sử dụng phương pháp ẩn dụ kết hợp Few-shot:

- *Mẫu 1 (Ẩn dụ):* Ví dụ về chiếc máy ATM (Người dùng chỉ thấy màn hình/nút bấm - Public Interface; không thấy bộ đếm tiền bên trong - Private State).
- *Mẫu 2 (Code thực tế):* Viết một class BankAccount có thuộc tính balance. Hãy chỉ ra hậu quả nếu balance là public (người dùng có thể sửa số dư trực tiếp) so với khi dùng phương thức deposit().
Requirement: Giải thích tại sao việc kiểm soát truy cập dữ liệu lại giúp code bền vững hơn (Robustness).

3. Tác vụ 3: Thiết kế công cụ kiểm tra tư duy (Quiz)

- **Prompt Cơ bản:** "Tạo 5 câu hỏi trắc nghiệm về Class và Object."
- **Prompt Cải tiến:** "Tạo 5 câu hỏi trắc nghiệm dựa trên slide bài giảng số 05. Câu hỏi phải bao quát được định nghĩa Class, Object và các Access Modifiers."

- **Prompt Nâng cao (Bloom's Taxonomy + Distractor Analysis):**

Task: Thiết kế bộ quiz 5 câu cho sinh viên UET với 3 cấp độ: Nhận biết (1 câu), Thông hiểu (2 câu), Vận dụng (2 câu).

Instruction:

1. Các phương án nhiễu phải được xây dựng từ những lỗi sai phổ biến của sinh viên năm nhất (ví dụ: nhầm lẫn Object là một kiểu dữ liệu thay vì một thực thể).
2. Sau mỗi câu hỏi, hãy cung cấp phần "Giải thích lý do" tập trung vào việc tại sao phương án đúng lại phản ánh đúng bản chất của OOP.
3. Sử dụng các đối tượng thực tế từ bài giảng như Student, Rectangle hoặc Circle.

III. Thử nghiệm và so sánh kết quả

1. Tác vụ 1:

- Prompt Cơ bản:

Hãy tóm tắt bài giảng Introduction to OOP này

Show thinking

Chào đại ca, dựa trên nội dung bài giảng số 05 về Lập trình hướng đối tượng (OOP) mà đại ca cung cấp, em xin tóm tắt lại những "bí kíp" cốt lõi nhất để đại ca nắm chắc kiến thức nhé.

1. Sự chuyển đổi tư duy: Từ Thủ tục sang Đối tượng

Đây là bước ngoặt quan trọng nhất trong cách lập trình:

- **Lập trình thủ tục (Procedural):** Tập trung vào hành động (Làm cái gì?). Dữ liệu và hàm tách rời nhau. Khi dự án lớn lên, việc quản lý dữ liệu trở nên cực kỳ rắc rối.
- **Lập trình hướng đối tượng (OOP):** Tập trung vào đối tượng (Ai làm?). Dữ liệu và các thao tác trên dữ liệu đó được đóng gói lại thành một thực thể duy nhất.

2. Khái niệm Lớp (Class) và Đối tượng (Object)

Đại ca cứ tưởng tượng thế này cho dễ hiểu:

- **Lớp (Class):** Là một cái khuôn hoặc bản thiết kế. Nó định nghĩa các đặc điểm và hành vi chung. Ví dụ: Lớp `SVinhVienUET`.
- **Đối tượng (Object):** Là một thực thể cụ thể được tạo ra từ cái khuôn đó. Ví dụ: "Dương" là một đối tượng cụ thể của lớp `SVinhVienUET`.

3. Các thành phần của một Lớp

Một lớp thường có hai thành phần chính (thường được gọi là các Member):

1. Thuộc tính (Attributes/Data Members): Các đặc điểm của đối tượng (ví dụ: tên, mã sinh viên, điểm GPA).
2. Phương thức (Methods/Member Functions): Các hành động mà đối tượng có thể thực hiện (ví dụ: học bài, đi thi, tính điểm trung bình).

4. Tính đóng gói (Encapsulation) & Che giấu dữ liệu

Đây là "linh hồn" của bài giảng này để đảm bảo chương trình an toàn và dễ bảo trì:

- **Access Modifiers (Phạm vi truy cập):**
 - `private`: Chỉ những hàm bên trong lớp đó mới được phép chạm vào dữ liệu. Đây là cách ta bảo vệ thuộc tính khỏi sự can thiệp sai trái từ bên ngoài.
 - `public`: Các hàm hoặc đối tượng bên ngoài có thể gọi và sử dụng.
- **Interface (Giao diện):** Chúng ta thường dùng các phương thức `public` (như Getter và Setter) để làm "cầu nối" cho phép bên ngoài tương tác với dữ liệu `private` một cách có kiểm soát.

```
graph TD; INHERITANCE[INHERITANCE  
Reusability] <--> POLYMORPHISM[POLYMORPHISM  
Flexibility]; POLYMORPHISM <--> ENCAPSULATION[ENCAPSULATION  
Security]; INHERITANCE <--> ENCAPSULATION; OOP((OOP))
```

5. Hàm khởi tạo (Constructor)

- Là một phương thức đặc biệt, tự động chạy ngay khi một đối tượng được tạo ra.
- Nó có tên trùng khít với tên Lớp và không có kiểu trả về.
- Nhiệm vụ chính là thiết lập giá trị ban đầu cho các thuộc tính của đối tượng.

Tóm lại: Bài giảng này muốn đại ca hiểu rằng trong OOP, chúng ta không còn nhìn chương trình như một dãy các bước thực hiện, mà nhìn nó như một hệ thống các đối tượng tương tác với nhau. Việc giữ cho dữ liệu an toàn (`private`) và chỉ tương tác qua các dịch vụ công khai (`public`) là chìa khóa để viết code chuẩn "UET".

- Prompt Cải tiến:

Tóm tắt bài giảng về OOP. Hãy tập trung vào sự khác biệt giữa Lập trình thủ tục và OOP. Trình bày dưới dạng danh sách so sánh.

Show thinking

Bài giảng về **Lập trình hướng đối tượng (OOP)** tập trung vào sự chuyển đổi từ duy từ việc viết mã theo trình tự các bước (Thủ tục) sang việc mô hình hóa thế giới thực thông qua các đối tượng.

Dưới đây là bảng so sánh chi tiết giữa **Lập trình thủ tục (Procedural Programming)** và **Lập trình hướng đối tượng (OOP)** để bạn dễ dàng nắm bắt:

Tiêu chí	Lập trình Thủ tục (Procedural)	Lập trình Hướng đối tượng (OOP)
Cách tiếp cận	Top-down: Chia bài toán lớn thành các hàm (chương trình con) nhỏ hơn.	Bottom-up: Thiết kế các đối tượng trước, sau đó kết hợp chúng để giải quyết bài toán.
Trọng tâm	Tập trung vào hành động (Làm cái gì? - What to do?).	Tập trung vào thực thể/dữ liệu (Ai làm? - Who does it?).
Cấu trúc mã	Chương trình là một danh sách các chỉ dẫn, hàm này gọi hàm kia.	Chương trình là một tập hợp các đối tượng tương tác với nhau qua các "thông điệp".
Dữ liệu & Hàm	Dữ liệu và hàm tách rời. Dữ liệu "trôi nổi" tự do và được truyền qua các hàm.	Dữ liệu và các hàm thao tác trên dữ liệu đó được đóng gói lại thành một Lớp (Class) .
Bảo mật	Thấp. Dữ liệu thường được để ở dạng toàn cục (global), dễ bị thay đổi ngoài ý muốn.	Cao. Dữ liệu được che giấu (<code>private</code>), chỉ có thể truy cập qua các phương thức công khai.
Khả năng tái sử dụng	Khó khăn. Thường phải sao chép mã nguồn khi muốn dùng lại logic ở nơi khác.	Rất cao nhờ tính Kế thừa (Inheritance) và các Lớp đã được module hóa.
Mở rộng/Bảo trì	Khó bảo trì khi dự án lớn vì các hàm có mối liên hệ chằng chéo phức tạp.	Dễ dàng quản lý nhờ tính module hóa; thay đổi một lớp ít ảnh hưởng đến các lớp khác.

Các khái niệm cốt lõi cần nhớ

Để hiểu sâu hơn về sự khác biệt này, bạn cần nắm vững "bộ tứ" của OOP được nhắc đến trong bài giảng:

- Tính Đóng gói (Encapsulation):** Gom dữ liệu và hành vi vào một "hộp đen". Bạn không cần biết bên trong cái đồng hồ hoạt động thế nào, bạn chỉ cần biết cách xem giờ qua mặt số (giao diện).
- Tính Trừu tượng (Abstraction):** Chỉ trình bày những thông tin cần thiết và ẩn đi những chi tiết triển khai phức tạp.
- Lớp (Class) vs. Đối tượng (Object):** Lớp là bản thiết kế (ví dụ: Bản vẽ xe hơi), còn Đối tượng là thực thể cụ thể được tạo ra từ bản thiết kế đó (ví dụ: Chiếc xe VinFast màu đỏ của bạn).

Lời khuyên nhỏ: Lập trình thủ tục giống như bạn tự tay làm mọi công việc theo một tờ hướng dẫn dài dằng dặc. Trong khi đó, OOP giống như việc bạn là một quản lý, giao việc cho các "chuyên gia" (đối tượng), mỗi chuyên gia biết tự quản lý dữ liệu và công việc của chính mình. Cách tiếp cận của OOP giúp code của bạn "sạch" và chuyên nghiệp hơn rất nhiều khi dự án trở nên phức tạp.

- Prompt Nâng cao:

Role: Bạn là một Senior Software Architect chuyên về hệ thống hướng đối tượng. Task: Hãy phân tích sự chuyển đổi từ Procedural sang OOP dựa trên nội dung bài giảng. Chain-of-Thought:...

Show thinking

Dưới góc độ của một Kiến trúc sư Phần mềm (Software Architect), việc chuyển đổi từ tư duy lập trình thủ tục sang hướng đối tượng không chỉ là thay đổi cú pháp, mà là một cuộc cách mạng về cách chúng ta quản lý độ **phức tạp** (complexity) của hệ thống.

Dưới đây là phân tích chuyên sâu về sự chuyển đổi này dựa trên nội dung bài giảng số 05.

1. Khủng hoảng của Lập trình Thủ tục: Sự tách rời chết người

Trong lập trình thủ tục (Procedural Programming), chúng ta coi thế giới là một tập hợp các **động tử**. Dữ liệu (Data) và Hàm (Functions) tồn tại ở hai thái cực độc lập:

- Dữ liệu thụ động (Passive Data):** Các cấu trúc dữ liệu (`struct`) chỉ là kho chứa thông tin đơn thuần, không có khả năng tự bảo vệ.
- Hàm chủ động (Active Functions):** Các hàm xử lý dữ liệu từ bên ngoài.
- Hệ quả (Side Effects):** Vì dữ liệu thường được để ở phạm vi toàn cục hoặc truyền tham chiếu rộng rãi, bất kỳ hàm nào cũng có thể can thiệp và thay đổi trạng thái dữ liệu một cách mất kiểm soát. Điều này dẫn đến hiện tượng "Spaghetti Code" – nơi một thay đổi nhỏ ở cấu trúc dữ liệu có thể làm sụp đổ toàn bộ hệ thống các hàm liên quan.

2. OOP và Kỹ thuật "Đóng gói": Hợp nhất Trạng thái và Hành vi

OOP giải quyết bài toán trên bằng cách giới thiệu khái niệm **Đối tượng (Object)**. Đây là một thực thể tự quản (Self-governing entity) kết hợp chặt chẽ giữa:

- Trạng thái (State):** Các thuộc tính dữ liệu.
- Hành vi (Behavior):** Các phương thức thao tác trên chính dữ liệu đó.

Cơ chế đóng gói (Encapsulation) đóng vai trò là "lớp vỏ bọc" bảo vệ:

- Sử dụng Access Modifiers (`private`) để "che giấu" chi tiết triển khai nội bộ.
- Chỉ phơi bày ra bên ngoài các giao diện công khai (`public interface`).

Lúc này, đối tượng không còn là một kho chứa dữ liệu thụ động nữa. Nó trở thành một **thực thể thống minh**, có trách nhiệm tự duy trì tính đúng đắn của dữ liệu bên trong chính nó.

3. Minh họa thực tế: Lớp `Student` (Sinh viên)

Hãy nhìn vào sự khác biệt khi chúng ta thiết kế một thực thể `Student` theo tư duy OOP:

```
C++class Student {private:    // Trạng thái (State) được bảo vệ - Không thể sửa đổi tùy tiện từ bên ngoài    string name;    string ID;    float gpa;public:    // Hàm khởi tạo (Constructor) - Thiết lập trạng thái ban đầu an toàn    Student(string n, string id) : name(n), ID(id), gpa(0.0) {}    // Hành vi (Behavior) - Đối tượng tự thực hiện hành động của mình    void study(float hours) {        // Logic xử lý nội bộ: học càng nhiều, gpa càng tăng (giả định)        gpa += hours * 0.1;        if (gpa > 4.0) gpa = 4.0;        cout << name << " (ID: " << ID << ") is studying hard!" << endl;    }};
```

Phân tích sự chuyển đổi:

- Trong lập trình thủ tục, đại ca sẽ có một hàm `study(student_data, hours)`. Nếu đại ca vô tình truyền vào một dữ liệu rác, hàm sẽ vẫn chạy và làm sai lệch kết quả.
- Trong OOP, đại ca gọi `student.study(5)`. Bản thân đối tượng `student` biết nó là ai, mã số gì và nó sẽ tự cập nhật điểm số của mình theo các quy tắc logic đã được định nghĩa sẵn trong lớp.

Kết luận của Kiến trúc sư

Sự chuyển đổi sang OOP giúp chúng ta xây dựng hệ thống theo dạng **Module**. Mỗi đối tượng là một viên gạch độc lập, có khả năng tự quản lý. Điều này không chỉ giúp mã nguồn dễ đọc hơn mà còn cực kỳ quan trọng trong việc bảo trì và mở rộng các hệ thống phần mềm quy mô lớn tại UET hay bất kỳ doanh nghiệp công nghệ nào.

2. Tác vụ 2:

- Prompt Cơ bản:

Giải thích tính đóng gói trong OOP

Show thinking

Tính đóng gói (Encapsulation) là một trong bốn trụ cột chính của Lập trình hướng đối tượng (OOP). Nếu coi một chương trình là một nhà máy, thì tính đóng gói chính là cách chúng ta thiết kế các chiếc máy sao cho công nhân chỉ cần biết bấm nút mà không cần (và không được phép) chạm tay vào hệ thống bánh răng bên trong.

Dưới đây là cách hiểu chi tiết và dễ nhớ nhất về khái niệm này:

1. Bản chất của Tính đóng gói

Tính đóng gói là việc gom nhóm các thuộc tính (dữ liệu) và phương thức (hành vi) có liên quan vào một đơn vị duy nhất gọi là **Lớp (Class)**. Đồng thời, nó ngăn chặn việc truy cập trực tiếp vào trạng thái nội bộ của đối tượng từ bên ngoài.

```
graph TD; INHERITANCE -- Reusability --> POLYMORPHISM; POLYMORPHISM -- Flexibility --> ENCAPSULATION; ENCAPSULATION -- Security --> INHERITANCE; OOP((OOP))
```

2. Cơ chế hoạt động: "Private" và "Public"

Để thực hiện tính đóng gói, chúng ta sử dụng các công cụ kiểm tra quyền truy cập (Access Modifiers):

- Dữ liệu (Private):** Các thuộc tính quan trọng (như mật khẩu, số dư tài khoản) thường được để ở chế độ `private`. Điều này có nghĩa là "người lạ" bên ngoài Class không thể tự ý sửa đổi chúng.
- Giao diện (Public):** Các phương thức cho phép tương tác với dữ liệu được để ở chế độ `public`. Đây là các "cửa ngõ" an toàn để bên ngoài có thể yêu cầu đối tượng thực hiện hành động hoặc lấy thông tin.

- Prompt Cải tiến:

Ví dụ thực tế: Khi bạn đi rút tiền ở cây ATM, bạn chỉ tương tác với màn hình và phím bấm (Public Interface). Bạn không thể mở thùng máy ra để tự đếm tiền bên trong (Private Data). Bạn yêu cầu máy rút tiền, và máy sẽ tự kiểm tra số dư và trừ tiền trong hệ thống của nó.

3. Getter và Setter: Những người gác cổng

Ví dữ liệu là `private`, chúng ta sử dụng hai loại phương thức đặc biệt để tương tác với chúng:

- Getter:** Dùng để đọc giá trị của thuộc tính.
- Setter:** Dùng để thay đổi giá trị. Tại đây, bạn có thể thêm các logic kiểm tra (Ví dụ: Không cho phép đặt `tuổi` là số âm).

4. Tại sao tính đóng gói lại quan trọng?

Lợi ích	Giải thích
Bảo mật dữ liệu	Ngăn chặn các tác động sai trái từ bên ngoài làm hỏng trạng thái của đối tượng.
Dễ bảo trì	Bạn có thể thay đổi cách triển khai bên trong một Class mà không làm ảnh hưởng đến những phần khác của chương trình.
Giảm sự phụ thuộc	Người dùng chỉ cần biết "máy làm được gì" chứ không cần biết "máy làm thế nào", giúp code gọn gàng hơn.
Tính linh hoạt	Bạn có thể kiểm soát dữ liệu chặt chẽ hơn (ví dụ: biến một thuộc tính thành chỉ đọc - Read-only).

Export to Sheets

Bạn đang tìm hiểu khái niệm này để áp dụng vào một ngôn ngữ lập trình cụ thể nào (như C++, Java, hay Python) không?

Giới thiệu Encapsulation là gì và cơ chế "chuẩn chỉ" cho dân kỹ thuật chúng ta cần Getter và Setter?"

Show thinking

Chào đại ca, để giới thiệu về Encapsulation (Tinh đóng gói) một cách "chuẩn chỉ" cho dân kỹ thuật, chúng ta sẽ đi từ bản chất lý thuyết đến thực hành code C++.

1. Encapsulation là gì?

Encapsulation là kỹ thuật bao bọc dữ liệu (variables) và các hàm thao tác trên dữ liệu đó (functions) vào một đơn vị duy nhất gọi là Lớp (Class).

Mục tiêu cốt lõi của nó là **Che giấu thông tin (Data Hiding)**: Ngăn chặn các thành phần bên ngoài can thiệp trực tiếp vào trạng thái nội bộ của đối tượng. Đại ca chỉ phải bày ra những gì cần thiết thông qua giao diện công khai (public interface).

Encapsulation

INHERITANCE

Reusability

OOP

POLYMORPHISM

Flexibility

ENCAPSULATION

Security

2. Ví dụ bằng code C++

Hãy tưởng tượng đại cơ đang viết code quản lý tài khoản ngân hàng. Nếu không có tính đồng bộ, ai cũng có thể sửa số dư của đại cơ thành 0 một cách con số âm vô lý.

```

C++
#include <iostream>
#include <string>

class BankAccount {
private:
    // thuộc tính để private để thực hiện Data Hiding
    std::string owner;
    double balance;

public:
    // Hàm khởi tạo (Constructor)
    BankAccount(std::string name, double initial_balance) {
        owner = name;
        if (initial_balance >= 0) {
            balance = initial_balance;
        } else {
            balance = 0;
            std::cout << "Số dư ban đầu không hợp lệ!" << std::endl;
        }
    }

    // GETTER: Cho phép xem số dư nhưng không cho sửa trực tiếp
    double getBalance() const {
        return balance;
    }

    // SETTER: Cho phép nạp tiền nhưng phải đi kèm điều kiện logic
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            std::cout << "Nạp thành công: " << amount << std::endl;
        } else {
            std::cout << "Số tiền nạp phải > 0!" << std::endl;
        }
    }
};

int main() {
    BankAccount myAcc("Duong Dai Co", 1000.0);

    // myAcc.balance = -500; // LỖI! Không thể truy cập vi balance là private
    myAcc.deposit(500); // Nạp 1k
    std::cout << "Số dư hiện tại: " << myAcc.getBalance() << std::endl;

    return 0;
}

```

3. Tại sao chúng ta cần Getter và Setter ?

Nhiều người mới học thường thắc mắc: "Sao không để `public` luôn cho nhanh, bày đặt dùng hàm làm gì?". Đây là lý do tại sao Getter và Setter là "tinh hồn" của chúng tôi:

A. Kiểm soát tính hợp lệ (Validation)

Nếu để `public`, người dùng có thể gán `age = -5` hoặc `gpa = 100`. Với Setter, đại ca có thể đặt chốt chặn:

```
if (value >= 0 && value <= 4.0) { gpa = value; }
```

B. Tính linh hoạt (Read-only hoặc Write-only)

- Muốn người dùng chỉ được xem mà không được sửa? Chỉ viết **Getter**.
- Muốn người dùng chỉ được đổi mật khẩu mà không được xem mật khẩu cũ? Chỉ viết **Setter**.

C. Dễ dàng bảo trì và thay đổi (Maintenance)

Giả sử sau này đại ca muốn thay đổi đơn vị tiền từ USD sang VND bên trong Class.

- Nếu dùng Getter:** Đại ca chỉ cần sửa logic trong hàm `getBalance()` (nhân thêm tỷ giá).
- Nếu để Public:** Đại ca phải đi tìm tất cả mọi nơi trong hàng nghìn dòng code có sử dụng biến `balance` để sửa lại. "Ác mộng" thực sự đấy đại ca!

D. Che giấu sự phức tạp (Abstraction)

Người dùng chỉ cần gọi `deposit()`, họ không cần biết bên trong đại ca tính toán thuế, phí hay ghi log vào database như thế nào.

Tóm lại: Encapsulation giúp code của đại ca "an toàn" và "sạch" hơn. Nó giống như việc đại ca lái xe máy: đại ca dùng tay ga (Interface) để chạy chứ không ai vịn vào vịn tay ga vào buồng đốt để điều chỉnh lượng xăng (Internal Data) cả!

Đại ca thấy phần giải thích này đã đủ để đưa vào bảo cáo chưa? Nếu cần thêm ví dụ về các lỗi thường gặp khi không dùng gói, cứ bảo em nhé!

A. Kiểm soát tính hợp lệ (Validation)

```
if (value >= 0 && value <= 4.0) { gpa = value; }
```

- Muốn người dùng chỉ được xem mà không được sửa? Chỉ viết **Getter**.
- Muốn người dùng chỉ được đổi mật khẩu mà không được xem mật khẩu cũ? Chỉ viết **Setter**.

Giả sử sau này đại ca muốn thay đổi đơn vị tiền tệ từ USD sang VND bên trong Class.

- #### D. Che giấu sự phức tạp (Abstraction)

Tóm lại: Encapsulation giúp code của đại ca "an toàn" và "sạch" hơn. Nó giống như việc đại ca lái xe máy: đại ca dùng tay ga (interface) để chạy chứ không ai vào chạy vừa thò tay vào buồng đốt để điều chỉnh lượng xăng (Internal Data) cả!

Đại ca thấy phần giải thích này đã đủ để đưa vào báo cáo chưa? Nếu cần thêm ví dụ về các lỗi thường gặp khi không đóng gói, cứ bảo em nhé!

Role: Bạn là một giảng viên PLILT nổi tiếng với cách dạy trực quan. Context: Sinh viên đang gặp khó khăn trong việc hiểu tính đóng gói là gì và tại sao phải để thuộc tính là private....

Show thinking

Chào đại ca và các "chiến thần" lập trình tại UET!

Tôi biết các em đang cảm thấy cái khái niệm **Tính đóng gói (Encapsulation)** này thật là... phiền phức. Kiểu như: "Tại sao tôi phải tự xây tường rào quanh nhà mình rồi lại phải tự chế chìa khóa (Getter/Setter) để vào nhà?".

Để tôi giải mã cho các em bằng cách "trực quan hóa" tư duy kỹ sư nhé.

1. Ấn dụ: Chiếc máy ATM và "Lớp vỏ thép"

Hãy tưởng tượng các em đang đứng trước một chiếc máy ATM giữa ngã tư Xuân Thủy.

- Public Interface (Giao diện công khai):** Đó là màn hình cảm ứng, các nút bấm, khe cắm thẻ. Đây là những thứ duy nhất máy ATM cho phép các em chạm vào. Nó được thiết kế để các em tương tác một cách an toàn.
- Private State (Trạng thái nội bộ):** Phía sau lớp vỏ thép 10cm là bộ đếm tiền, các khay chứa tiền mặt, và hệ thống máy tính kết nối trực tiếp với ngân hàng.

Chuyện gì xảy ra nếu mọi thứ đều là **public**?

Nếu không có tính đóng gói, chiếc ATM sẽ không có vỏ thép. Bất cứ ai đi ngang qua cũng có thể thò tay vào khay tiền, tự đếm lấy 5 triệu rồi bỏ đi mà không cần thẻ. Đó không phải là một chiếc máy, đó là một thảm họa tài chính!

Trong lập trình, việc để thuộc tính là **private** chính là việc chúng ta "đúc vỏ thép" cho đối tượng để bảo vệ dữ liệu bên trong khỏi những tác động sai trái.

3. Tại sao kiểm soát truy cập tạo nên "Code Bền Vững" (Robustness)?

Tại sao các kỹ sư phần mềm tại các tập đoàn lớn lại tận tụy thời gian đóng gói? Câu trả lời nằm ở hai chữ **Robustness** (Tính chặt chẽ/bền vững):

- Chống lại sự "hồng hộc" dữ liệu (Data Integrity):** Khi để **private**, đối tượng nắm toàn quyền kiểm soát trạng thái của mình. Nó đảm bảo rằng dữ liệu không bao giờ rơi vào trạng thái "vô lý" (như số dư âm, tuổi -1, hay điểm GPA > 4.0).
- Khả năng bảo trì linh hoạt (Easy Maintenance):** Giả sử một ngày ngân hàng đổi cách tính phí giao dịch. Nếu để **public**, đại ca phải đi tìm hàng nghìn chỗ trong code có dùng **balance** để sửa. Còn với đóng gói, đại ca chỉ cần sửa đúng một chỗ bên trong hàm **deposit()**. Thế giới bên ngoài không hề hay biết, nhưng hệ thống vẫn hoạt động hoàn hảo.
- Che giấu sự phức tạp (Abstraction):** Người dùng chỉ cần biết gọi hàm **deposit()**, họ không cần biết bên trong máy đếm tiền theo kiểu gì, kết nối server ra sao. Code càng ít biết về nhau, hệ thống càng ít lỗi dây chuyền.

Lời kết của thầy: Đừng coi **private** là sự gò bó. Hãy coi nó là **lời cam kết** của một lập trình viên chuyên nghiệp: *"Tôi đảm bảo đối tượng này luôn an toàn và hoạt động đúng, bất kể các bạn dùng nó như thế nào!"*.

Đại ca thấy cách giải thích này đã đủ để "đả thông kinh mạch" cho các bạn sinh viên chưa? Nếu cần thêm một ví dụ về tính "Kế thừa" hay "Đa hình", cứ hô em một tiếng nhé!

2. So sánh Code thực tế: Lớp **BankAccount**

Hãy xem sự khác biệt giữa một thiết kế "ngây thơ" và một thiết kế "chuyên nghiệp".

Trường hợp 1: Thảm họa khi để **public**

Nếu biến **balance** (số dư) là công khai, bất kỳ đoạn code nào ở tận đầu đầu cũng có thể nhảy vào sửa nó.

```
C++
class BankAccount {
public:
    double balance; // Sai lầm chí mạng: Cửa mở toang cho kẻ trộm!
};

// Ở hàm main:
BankAccount myAcc;
myAcc.balance = 1000.0;
myAcc.balance = -500000.0; // HỦY QUÁ: Tài khoản có số dư âm vô lý nhưng code vẫn chạy.
```

Trường hợp 2: Bảo mật với **private và **deposit()****

Bằng cách dùng **private**, chúng ta ép mọi tương tác phải đi qua một "cổng kiểm soát" duy nhất.

```
C++
class BankAccount {
private:
    double balance; // Đã khóa chặt trong hàm
public:
    BankAccount() : balance(0) {}

    // Phương thức gửi tiền - Đây chính là "mặt bôn" trên ATM
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            cout << "Gửi tiền thành công!" << endl;
        } else {
            // Kiểm soát logic: Không cho phép gửi số tiền âm
            cout << "Lỗi: Số tiền gửi phải lớn hơn 0!" << endl;
        }
    }

    double getBalance() const { return balance; }
};

// Ở hàm main:
BankAccount myAcc;
// myAcc.balance = -100; // LỖI BIẾN DỊCH: "Cánh cửa thép" đã chặn bạn lại!
myAcc.deposit(500);      // Hợp lệ
myAcc.deposit(-200);     // Bị chặn bởi logic bên trong.
```

3. Tác vụ 3:

- Prompt Cơ bản:

Ôn tập: Class và Object (Lecture 05)

1 / 5

1. Trong lập trình hướng đối tượng, khái niệm 'Lớp' (Class) thường được so sánh với hình ảnh nào sau đây?

- A. Một biến số dùng để lưu trữ một giá trị duy nhất.
- B. Một ngôi nhà đã hoàn thiện và có người ở.
- C. Một bản vẽ kỹ thuật hoặc khuôn mẫu thiết kế.
- D. Một danh sách các lệnh thực hiện theo thứ tự.

Show hint

2. Khi chúng ta nói 'Dương' là một sinh viên của UET', trong ngữ cảnh OOP, 'Dương' đóng vai trò là gì?

- A. Một Lớp (Class).
- B. Một Phương thức (Method).
- C. Một Đối tượng (Object/Instance).

✓ That's right!
Dương là một thể hiện cụ thể, có dữ liệu riêng biệt được tạo ra từ lớp SinhVien.

- D. Một Thuộc tính (Attribute).

Show hint

3. Thành phần nào của một Lớp quy định các 'hành vi' hoặc 'khả năng' mà đối tượng có thể thực hiện?

- A. Access Modifiers (public/private).
- B. Các biến thành viên (Data Members).
- C. Các hàm thành viên (Member Functions/Methods).

✓ That's right!
Phương thức định nghĩa những gì đối tượng có thể làm (ví dụ: tínhDienTich(), hienThiThongTin()).

- D. Tên của Lớp.

Show hint

4. Tại sao trong bài giảng số 05 lại khuyến nghị để các thuộc tính (Attributes) ở trạng thái 'private'?

- A. Để chương trình chạy nhanh hơn và tiết kiệm bộ nhớ.
- B. Để ngăn chặn các hàm bên ngoài thay đổi dữ liệu một cách tùy tiện và sai logic.

✓ That's right!
Đây là cốt lõi của tính đóng gói, giúp bảo vệ tính toàn vẹn của dữ liệu nội bộ.

- C. Vì C++ không cho phép khai báo thuộc tính ở trạng thái public.
- D. Để các đối tượng khác có thể truy cập dữ liệu dễ dàng hơn.

Show hint

5. Điều gì phân biệt rõ rệt nhất giữa hai đối tượng khác nhau nhưng được tạo ra từ cùng một Lớp?

- A. Chúng có các bộ phương thức (hành vi) khác nhau.
- B. Chúng có các giá trị thuộc tính (trạng thái) riêng biệt trong bộ nhớ.

✓ That's right!
Mỗi đối tượng có một vùng nhớ riêng để lưu trữ giá trị các thuộc tính của chính nó.

- C. Chúng có tên lớp khác nhau.
- D. Chúng được viết bằng các ngôn ngữ lập trình khác nhau.

Show hint

- Prompt Cải tiến:

1. Trong lập trình hướng đối tượng, mối quan hệ giữa Class và Object được ví như điều gì sau đây?

A. Class là bản thiết kế (Blueprint), còn Object là ngôi nhà thực tế được xây dựng từ bản thiết kế đó.

✓ Right answer

Class đóng vai trò là khuôn mẫu định nghĩa các đặc tính, trong khi Object là một thực thể cụ thể được tạo ra từ khuôn mẫu đó.

B. Object là bản thiết kế, còn Class là sản phẩm thực tế.

C. Class và Object là hai tên gọi khác nhau của cùng một khái niệm.

D. Class là một tập hợp các Object có cùng giá trị thuộc tính.

✗ Not quite

Các Object từ cùng một Class có cấu trúc giống nhau nhưng giá trị thuộc tính (trạng thái) thường khác nhau.

2. Thành phần nào của một Class đại diện cho 'hành vi' (behavior) của đối tượng?

A. Biến tĩnh (Static variables)

B. Phương thức (Methods)

✓ That's right!

Các phương thức định nghĩa những gì mà đối tượng có thể thực hiện, tương ứng với hành vi trong thế giới thực.

C. Access Modifiers

D. Thuộc tính (Attributes)

Show hint

3. Để thực hiện nguyên tắc 'Che giấu dữ liệu' (Data Hiding), các thuộc tính trong một Class nên được khai báo với Access Modifier nào?

A. public

B. private

✓ Right answer

Khai báo private giúp ngăn chặn việc truy cập trực tiếp vào thuộc tính từ bên ngoài lớp, bảo vệ tính toàn vẹn của dữ liệu.

C. default (package-private)

✗ Not quite

Mặc định vẫn cho phép các lớp trong cùng package truy cập, không đảm bảo tính đóng gói tuyệt đối.

D. protected

4. Mục đích chính của việc sử dụng các phương thức Getter và Setter là gì?

A. Để biến các thuộc tính private thành public.

B. Để cho phép các lớp khác có thể sửa đổi thuộc tính private một cách có kiểm soát.

✓ Right answer

Thông qua Setter, ta có thể thêm các logic kiểm tra dữ liệu đầu vào trước khi thay đổi giá trị thuộc tính.

C. Để tăng tốc độ thực thi của chương trình.

✗ Not quite

Getter/Setter thực tế làm chương trình tốn thêm một chút thời gian gọi hàm, mục tiêu của chúng không phải là tốc độ.

D. Để thay thế hoàn toàn việc sử dụng Constructor.

Show hint

5. Trong các phát biểu sau về Access Modifiers, phát biểu nào là ĐÚNG?

A. Lớp con không thể truy cập vào các thành phần protected của lớp cha.

B. Biến private có thể được truy cập trực tiếp bởi các Object của lớp khác trong cùng package.

✗ Not quite

Private chỉ cho phép truy cập nội bộ bên trong chính lớp đó, bất kể package nào.

C. Mặc định (nếu không viết gì) thì mọi thành phần đều là private.

D. Một phương thức public có thể được gọi từ bất kỳ đâu trong ứng dụng.

✓ Right answer

Từ khóa public mở rộng phạm vi truy cập ra toàn bộ hệ thống miễn là lớp chứa nó có thể truy cập được.

- Prompt Nâng cao:

Mức độ 1: Nhận biết (1 câu)

Câu 1: Trong lớp `Student` được thiết kế để quản lý sinh viên UET, phát biểu nào sau đây mô tả đúng nhất về `Class`?

A. Là một sinh viên cụ thể đang ngồi trong giảng đường G2.

B. Là một biến kiểu cấu trúc (struct) dùng để lưu trữ tên và mã số sinh viên.

C. Là một "bản thiết kế" định nghĩa các thuộc tính (tên, MSV) và hành vi (đăng ký môn học) chung cho tất cả sinh viên.

D. Là một danh sách (Array/Vector) chứa tất cả các đối tượng sinh viên trong một lớp học.

- Giải thích lý do:** Đáp án C phản ánh đúng bản chất của OOP. Class không phải là một thực thể tồn tại trong bộ nhớ khi chạy (như A), cũng không đơn thuần là một kiểu dữ liệu tĩnh (như B). Nó là một **khung mẫu trừu tượng**. Việc nhầm lẫn Class là một "tập hợp" (D) là lỗi phổ biến khi sinh viên tư duy theo kiểu quản lý danh sách thay vì tư duy theo kiểu thực thể.

Mức độ 2: Thông hiểu (2 câu)

Câu 2: Giả sử đại ca có dòng lệnh: `Rectangle r1 = new Rectangle();`. Khẳng định nào sau đây về `r1` là SAI?

A. `r1` là một đối tượng (Object) cụ thể của lớp `Rectangle`.

B. `r1` đóng vai trò là một kiểu dữ liệu mới do người dùng định nghĩa.

C. `r1` chiếm một vùng không gian nhất định trong bộ nhớ (Heap) để lưu trữ các giá trị chiều dài, chiều rộng.

D. `r1` có trạng thái (giá trị thuộc tính) và hành vi (phương thức tính diện tích) riêng.

- Giải thích lý do:** Đáp án B sai vì `Rectangle` mới là kiểu dữ liệu, còn `r1` là biến (hoặc tham chiếu) trỏ đến một thực thể. Sinh viên UET mới học thường nhầm lẫn giữa "kiểu" (Type) và "biến" (Variable/Instance). OOP tách biệt rõ ràng: Class là kiểu, Object là thực thể có định danh riêng.

Câu 3: Tại sao trong lớp `Circle`, thuộc tính `private double radius;` lại được ưu tiên hơn so với `public double radius;`?

A. Để chương trình chạy nhanh hơn và tốn ít bộ nhớ hơn.

B. Để ngăn chặn hoàn toàn việc người dùng truy cập hoặc thay đổi giá trị bán kính.

C. Để đảm bảo tính đóng gói, buộc việc thay đổi bán kính phải thông qua kiểm soát (ví dụ: bán kính không được âm).

D. Vì ngôn ngữ Java bắt buộc tất cả thuộc tính phải là private.

- Giải thích lý do:** Đáp án C phản ánh bản chất của **Encapsulation**. Nhiều sinh viên chọn B vì nghĩ private là "cấm cửa", nhưng thực tế nó chỉ là "kiểm soát cửa". Mục tiêu là bảo vệ tính toàn vẹn của đối tượng, tránh việc một đối tượng `Circle` có bán kính bằng -5 (điều vô lý về mặt logic nhưng compiler vẫn chấp nhận nếu để public).

Mức độ 3: Vận dụng (2 câu)

Câu 4: Đại ca đang viết phương thức `setAge(int age)` cho lớp `Student`. Đoạn mã nào sau đây thể hiện đúng tinh thần của OOP nhất?

A. `public void setAge(int age) { this.age = age; }`

B. `public void setAge(int a) { if (a > 17 && a < 100) age = a; }`

C. `private void setAge(int age) { this.age = age; }`

D. Không cần viết phương thức này, cứ để thuộc tính `age` là public cho tiện.

- Giải thích lý do:** Đáp án B là cách vận dụng đúng. Việc kiểm tra điều kiện (`a > 17`) chính là việc thực thi **logic nghiệp vụ** ngay tại cửa ngõ của đối tượng. Sinh viên thường chọn A vì lười hoặc chưa hiểu rằng Setter sinh ra là để làm "người gác cổng", chứ không chỉ để gán giá trị thuần túy.

Câu 5: Cho lớp `Rectangle` có hai thuộc tính `private double width, height;`. Để một lớp khác có thể tính diện tích của một hình chữ nhật cụ thể mà vẫn giữ đúng quy tắc đóng gói, đại ca nên làm gì?

A. Chuyển `width` và `height` sang `public` để lớp kia tự lấy về nhân với nhau.

B. Viết phương thức `public double getArea()` bên trong lớp `Rectangle` để tự tính và trả về kết quả.

C. Viết một hàm tính diện tích ở lớp bên ngoài và truyền đối tượng `Rectangle` vào làm tham số.

D. Copy mã nguồn của lớp `Rectangle` vào lớp đang cần tính.

- Giải thích lý do:** Đáp án B là lựa chọn tối ưu. Trong OOP, đối tượng nào nắm giữ dữ liệu thì đối tượng đó nên có trách nhiệm xử lý dữ liệu đó (**Expert Pattern**). Việc tính diện tích là hành vi của hình chữ nhật, không nên đẩy trách nhiệm đó ra ngoài (C) hay phá vỡ tính đóng gói (A).

4. So sánh các phiên bản prompt một cách tổng quát:

Phiên bản Prompt	Đặc điểm phản hồi của AI	Điểm mạnh	Điểm yếu
Cơ bản	Văn bản dàn trải, chỉ nhắc lại các định nghĩa trong slide một cách máy móc.	Nhanh, ngắn gọn.	Thiếu sự kết nối logic, không giải thích được "tại sao".
Cải tiến	Đã có cấu trúc bảng biểu, các ví dụ code C++ cơ bản.	Dễ theo dõi, sát với yêu cầu bài tập.	Vẫn còn thiếu tính thực tiễn và chiều sâu tư duy

Phiên bản Prompt	Đặc điểm phản hồi của AI	Điểm mạnh	Điểm yếu
			hệ thống.
Nâng cao	Phản hồi sắc sảo, sử dụng thuật ngữ chuyên môn kết hợp ẩn dụ cực kỳ dễ hiểu. Code minh họa có giải thích từng dòng logic.	Tạo ra sự thấu hiểu (Insights). Phù hợp để làm tài liệu học tập chuyên sâu.	Đòi hỏi người viết prompt phải có kiến thức nền để kiểm soát.

IV. Phân tích hiệu quả của Prompt

Lý do Prompt Nâng cao đạt hiệu quả vượt trội:

1. **Thiết lập vai trò (Role-playing):** Khi gán vai "Senior Architect", AI sẽ ưu tiên các tokens liên quan đến tính bền vững, kiến trúc hệ thống thay vì chỉ liệt kê cú pháp.
2. **Kỹ thuật Chain-of-Thought:** Buộc LLM phải thực hiện các bước suy luận trung gian. Ví dụ, trước khi giải thích Encapsulation, nó phải giải thích vấn đề của lập trình thủ tục. Điều này tạo ra một mạch logic không thể tách rời.
3. **Few-shot Learning:** Bằng cách đưa ra mẫu ẩn dụ (ATM) và mẫu code (BankAccount), chúng ta đã "neo" (anchor) tư duy của AI vào một hướng cụ thể, giúp kết quả đầu ra không bị lệch lạc hoặc quá hàn lâm.
4. **Kiểm soát nhiễu thông tin:** Việc sử dụng các Constraints (Ràng buộc) giúp loại bỏ các câu trả lời rườm rà, tập trung thẳng vào trọng tâm bài giảng số 05.

V. Tổng hợp nguyên tắc và mẹo viết prompt.

Để trở thành một "bậc thầy" Prompt Engineering trong học tập, cần tuân thủ 5 nguyên tắc vàng:

1. **Nguyên tắc "Cụ thể hóa":** Thay vì nói "tóm tắt bài giảng", hãy nói "tóm tắt 3 ý chính quan trọng nhất dành cho người chuẩn bị thi giữa kỳ".

2. **Nguyên tắc "Phân lớp kiến thức":** Luôn yêu cầu AI giải thích theo cấp độ (Từ ẩn dụ đời thực à Lý thuyết toán học/logic à Triển khai mã nguồn).
3. **Nguyên tắc "Đối chiếu":** Yêu cầu AI so sánh cái mới (OOP) với cái cũ (Procedural) để làm nổi bật ưu điểm.
4. **Sử dụng ký hiệu chuyên dụng:** Luôn yêu cầu đầu ra code trong khối Markdown và công thức trong LaTeX để đảm bảo độ chính xác kỹ thuật.
5. **Vòng lặp phản hồi (Feedback Loop):** Nếu AI trả lời chưa ưng ý, hãy dùng lệnh: "Phần giải thích về "private" còn hơi mơ hồ, hãy lấy thêm ví dụ về việc truy cập trái phép bộ nhớ để làm rõ hơn."